Oracle AWR Report

Developer & Application Reference Guide

A practical guide to reading AWR reports, understanding wait events, SQL statistics, segment activity, and RAC metrics - with developer-focused triage patterns and checklists.

Version 1.0 | May 2026 | Internal Reference

Oracle AWR Report - Developer & Application Reference Guide

Purpose: A practical reference for developers, application owners, and support engineers who need to read an Automatic Workload Repository (AWR) report, understand what each section means, and decide what action to take.

Audience: Java/JDBC application developers, STOV/STOALT support, performance triage teams.

Version: 1.0 | May 2026

Table of Contents

1. What Is AWR?

The **Automatic Workload Repository (AWR)** is Oracle's built-in performance data warehouse. Every 60 seconds (default), Oracle snapshots hundreds of metrics: SQL executions, wait events, I/O, memory, segment access, and session activity.

An **AWR report** compares two snapshots (start and end) and produces a human-readable summary of what the database spent time doing during that window.

Why developers should care

* Your SQL, PL/SQL packages, and JDBC calls appear by **SQL_ID** in the report.
* Lock contention, excessive I/O, and parse overhead often trace back to **application design or batch timing**.
* AWR tells you **where time went**, not always **why** - but it narrows investigation dramatically.

AWR vs other tools

Tool	What it shows	Best for
AWR report	Aggregated stats over minutes/hours	Post-incident analysis, capacity review
ASH (Active Session History)	Sampled session activity every second	Pinpointing blockers, short spikes
ADDM	Oracle's automated recommendations	High-level diagnosis from AWR data
V$ views / ASH live	Real-time	Active troubleshooting
SQL Monitor / Real-Time SQL	Single SQL execution detail	Long-running or parallel SQL

2. Before You Open the Report

Choose the right snapshot window

* Align the AWR window with the **incident time** (user-reported slowness, batch failure, timeout).
* Include **5-10 minutes before** the incident if possible - shows buildup.
* Avoid windows that span unrelated workloads (e.g., nightly batch + business hours mixed).

Know your baseline

* A report without comparison is harder to interpret. Ask DBA for a **similar window on a normal day**.
* Key baseline numbers: **AAS**, **DB Time**, top wait class %, top SQL IDs.

Gather context

Before reading, note:

* What application action was slow? (Save STOALT, export, login, search)
* Single user or many users?
* RAC or single instance?
* Any batch job, MV refresh, or deployment at that time?

3. Report Header & Snapshot Metadata

The top of every AWR report contains:

Field	Meaning
DB Name / DB Id	Which database (confirm you have the right environment)
Instance	Instance name(s) - important in RAC
Release	Oracle version (features and wait event names vary by version)
RAC	Yes/No - unlocks cluster-specific sections
Snapshot Begin / End	Exact time window
Elapsed Time	Wall-clock minutes between snapshots
DB Time	Total time all sessions spent active in the DB (can exceed elapsed in parallel/multi-session workloads)
CPUs / Cores / Sockets	Hardware capacity
Load Average	OS-level CPU queue (high = CPU saturation or runnable processes)

Critical derived metric: AAS (Average Active Sessions)

```
AAS = DB Time / Elapsed Time
```

Interpretation:

* **AAS ~ number of CPU cores:** Database is reasonably busy, mostly compute-bound.
* **AAS >> CPU count:** Sessions are waiting (I/O, locks, latches) - bottleneck is not raw CPU.

* **AAS << 1:** Light load; slowness may be client/network or a few blocked sessions.

Example: Elapsed 480 min, DB Time 8868 min -> AAS ~ 18.5. If the server has 16 cores, the DB was consistently oversubscribed with waiting work.

4. Database Summary & Core Metrics

This section summarizes overall health:

* **DB Time vs Elapsed:** How much work happened vs wall clock.
* **% Total CPU:** How much of DB Time was on CPU vs waiting.
* **% IO Wait:** Time waiting for storage.
* **% Other Wait:** Locks, latches, cluster, network.

Red flags for developers

Pattern	Likely cause
High **User I/O** %	Full scans, missing indexes, parallel reads, MV refresh
High **Concurrency** %	Row/table locks, buffer busy, ITL waits
High **Commit** %	Frequent commits, log sync waits, small transaction batches
High **Cluster** % (RAC)	Cross-instance block transfers, hot blocks
High **Application** wait	User-defined waits (often batch/job frameworks)

5. Load Profile

Load Profile shows **rates per second** averaged over the snapshot window:

Metric	What it measures	Developer relevance
DB Time(s)	Active session seconds per wall second	Overall intensity
DB CPU(s)	CPU seconds per wall second	Compute intensity
Redo size (bytes)	Transaction volume	Bulk DML, batch jobs
Logical reads (blocks)	Buffer cache reads	SQL efficiency
Physical reads (blocks)	Disk reads	Cache miss, large scans
Executions (SQL)	SQL executions/sec	Chatty apps, missing bind reuse
Parses (SQL)	Parse calls/sec	Hard parse problems if high vs executions
Hard parses (SQL)	Full parse/sec	Literal SQL, no bind variables
User calls	OCI/JDBC calls/sec	Application call pattern
Transactions	Commits + rollbacks/sec	Transaction granularity

What to look for

* **High logical reads with low executions:** Few SQL statements reading too much data (bad plan or missing filter).
* **High hard parses:** Application not using bind variables or has excessive dynamic SQL.
* **High transactions with small redo:** Many tiny commits - consider batching.

6. Wait Classes

Wait classes group individual wait events into categories. The **% DB Time** column is your primary navigation aid.

Standard wait classes

Wait Class	Typical meaning	Application angle

DB CPU	On-CPU processing	SQL complexity, PL/SQL loops, inefficient algorithms
User I/O	Waiting for data blocks from disk (db file sequential/scattered read, direct path read)	Full table scans, index range scans on large tables, exports
System I/O	Background I/O (checkpoint, archiver)	Usually DBA/infrastructure; can spike during backup
Commit	Log file sync after COMMIT	Too-frequent commits; storage latency
Concurrency	Locks, latches, buffer busy	Hot rows, poor indexing, parallel DML conflicts
Configuration	Resource limits (processes, sessions)	Connection pool too large
Administrative	DBA operations	Index rebuild, stats gather during business hours
Application	User-defined (DBMS_LOCK, etc.)	Custom job coordination
Cluster	RAC interconnect (gc cr/current block)	Hot blocks on shared tables
Network	SQL*Net more data/from client	Large result sets, chatty round trips

Reading the pie chart / bar

If **one class dominates (>50%)**, start investigation there:

* **User I/O 60%** -> SQL and segment stats sections next.
* **Concurrency 20%+** -> Look for `enq: TX - row lock contention`, buffer busy waits.
* **Cluster significant on RAC** -> Identify hot objects in segment stats + ASH.

7. Top Wait Events

Individual wait events explain **exactly** what sessions waited on. Sorted by **% DB Time**.

High-value columns

Column	Meaning
Event	Wait event name
Waits	Number of wait occurrences
Total Wait Time (s)	Cumulative seconds waiting
Avg Wait (ms)	Average wait per occurrence
% DB time	Share of total DB time

Events developers see most often

I/O related

* **`db file sequential read`** - Index lookups, single-block reads. High avg wait -> storage latency or cold cache.
* **`db file scattered read`** - Multi-block reads (full scan). Often large table scans.
* **`direct path read`** - Parallel query, temp, or bypass buffer cache reads. Common in **PX workers** during exports/aggregations.
* **`direct path write temp`** - Sort/hash spill to temp tablespace.

Lock related

* **`enq: TX - row lock contention`** - Session A holds row lock, Session B waits. **Classic ORA-00060 scenario in AWR.** Trace to blocking SQL_ID via ASH.
* **`buffer busy waits` / `read by other session`** - Hot block contention (many sessions want same block).

Commit related

* **`log file sync`** - Commit wait for redo flush. High avg ms -> redo log or storage issue, or too many commits.

RAC related

* **`gc cr block busy` / `gc current block busy`** - Cross-instance block ping-pong on hot rows/blocks.

Client related

* **`SQL*Net more data from client`** - DB waiting for client to send more (large binds, slow app).
* **`SQL*Net message from dblink`** - Remote database link calls.

How avg wait helps

Avg wait	Interpretation
< 10 ms (I/O)	Often normal; high *count* is the issue
> 20 ms (I/O)	Storage or SAN latency
> 1 sec (locks)	Real blocking chains - find blocker in ASH
> 10 ms (log file sync)	Commit/redo path problem

8. SQL Statistics Sections

Multiple SQL sections sort the same SQL by different dimensions. Always note the **SQL_ID** - it is stable until the SQL is flushed or changed.

Key sections

Section	Sorted by	Use when
SQL ordered by Elapsed Time	Total time (all executions)	"What hurt most overall?"
SQL ordered by CPU Time	CPU consumption	Compute-heavy SQL
SQL ordered by User I/O Wait Time	I/O wait	Disk-bound SQL
SQL ordered by Gets	Logical reads	Buffer cache churn
SQL ordered by Physical Reads	Disk blocks read	Full scans, cache misses
SQL ordered by Executions	Call count	Chatty SQL
SQL ordered by Parse Calls	Parse count	Bind/literal problems
SQL ordered by Version Count	Child cursors	Shared pool pressure, bind mismatch

Important columns per SQL entry

Column	Meaning
Elapsed Time (s)	Total time for all executions in window
Executions	How many times it ran
Elapsed per Exec (s)	Average - compare across SQL
%Total	Share of that section's total
CPU Time	On-CPU portion
Buffer Gets	Logical reads per execution (efficiency indicator)
Physical Reads	Disk reads
Rows Processed	Output/scan volume
Module / Action	JDBC application tagging (if set)
SQL Text	Truncated statement - get full text from DBA

Mapping SQL_ID to your application

```
-- Full SQL text
SELECT sql_fulltext FROM v$sql WHERE sql_id = '&sql_id';

-- Recent sessions running it
SELECT sid, serial#, username, program, module, action, sql_id
FROM v$session WHERE sql_id = '&sql_id';

-- ASH: who blocked whom (if ASH available)
SELECT sample_time, session_id, session_serial#, sql_id, event, blocking_session
```

```
FROM dba_hist_active_sess_history
WHERE sql_id = '&sql_id'
  AND sample_time BETWEEN &begin AND &end;
```

Developer action matrix

SQL pattern	Action
High elapsed, high gets, low rows	Bad plan - check indexes, stats, filters
High executions, moderate elapsed each	Reduce call frequency, batch operations
High physical reads	Avoid full scan; partition pruning; MV vs live table
High CPU, low I/O	PL/SQL logic, regex, UDF in SELECT - refactor
Same SQL many versions	Bind variable mismatch - standardize literals

9. Instance Activity & System Statistics

These sections show **database-wide counters** and rates:

* **Instance Activity Stats:** db block changes, consistent gets, physical reads/writes, recursive calls, user commits/rollbacks.
* **System Statistics (ordered by statistic):** Detailed breakdown (e.g., `cell physical IO bytes`, `parse count (hard)`).

Useful stats for developers

Statistic	Indicates
db block changes	DML volume - which segments change most (cross-ref segment stats)
user commits	Transaction frequency
execute count	SQL execution volume
parse count (hard)	SQL not reused - literal SQL problem
physical reads direct	Direct path operations (parallel, LOB, temp)
redo size	Write volume

10. I/O Statistics

File I/O Stats

Shows I/O by **tablespace / datafile**: reads, writes, avg read/write ms.

* High avg read ms on specific tablespace -> storage hotspot or cold data.
* Temp tablespace activity -> sorts/hash joins spilling.

Tablespace & File I/O

Developers: identify if your schema's tablespaces dominate reads. Large export/query against one tablespace will show here.

I/O Profile (Exadata/cell aware environments)

If present, distinguishes **flash cache vs disk** - mostly DBA territory but confirms storage vs SQL issue.

11. Segment Statistics (Tables & Indexes)

One of the most actionable sections for application teams.

Segments listed by:

* Logical reads
* Physical reads
* Buffer busy waits
* DB block changes
* Row lock waits
* ITL waits

Columns to focus on

Column	Meaning
Object	OWNER.TABLE or INDEX name
%Total	Share of that metric in the instance
Statistic	What kind of activity

Interpretation

Dominance pattern	Likely issue
Table high logical reads	Heavily queried - MV, missing index, or cache-friendly scan
Table high db block changes	Heavy UPDATE/INSERT/DELETE target
Table high row lock waits	**Hot row updates** - many sessions updating same key
Index high logical reads	Index range scan on large index or index not selective
Index high buffer busy	Right-hand index insert hot spot (sequence/ID)

Linking to your code

If `ATOSM_O.SITES` dominates row lock waits and your app does `UPDATE SITES WHERE NAME = :B1`, you have evidence the table is a contention point - consider:

* Reducing transaction hold time
* Avoiding unnecessary updates to same site row
* Ordering lock acquisition consistently across modules

12. Memory, Buffer Cache & PGA

Buffer Pool Advisory

Shows **estimated physical reads** if buffer cache were larger/smaller. If DBA says "increase SGA" - this supports it. Developers rarely change this but explain if SQL is inherently scan-heavy.

Memory Statistics / SGA breakdown

* **Shared Pool:** Parses, PL/SQL, dictionary cache
* **Buffer Cache:** Data blocks
* **PGA:** Sorts, hash joins per session

High **PGA allocated** + temp I/O -> large sorts in your SQL (missing index for ORDER BY/GROUP BY).

Library Cache / Row Cache

High reloads/invalidations -> DDL during business hours or stats job causing cursor invalidation.

13. RAC-Specific Sections

When **RAC = YES**, additional sections appear:

Global Cache Transfer Statistics

* **gc cr/current block receive time** - Interconnect latency for consistent/current blocks.
* High cluster wait % + hot segment in segment stats -> block ping-pong across nodes.

Interconnect / Global CR Served

Mostly infrastructure. Developers care when **same hot row** is updated from sessions on different nodes.

Practical RAC guidance for apps

* Sticky sessions or partition workload by node where possible.
* Avoid many small updates to **same primary key** from random nodes.
* Sequence/cache settings on hot indexes - DBA + dev coordination.

14. Active Session History (ASH) & ADDM

ASH in AWR

Some reports include ASH sections or you generate **ASH report** for same window. ASH samples active sessions every second:

* **Session ID, Module, Program** - maps to JDBC pool / application server
* **SQL_ID** - what was running
* **Event** - what it waited on
* **Blocking Session** - who blocked whom

Use ASH when: AWR shows `enq: TX - row lock` but you need the **blocker SQL_ID and username**.

ADDM (Automatic Database Diagnostic Monitor)

Oracle's narrative recommendations at end of AWR or separate ADDM report:

* "Top SQL statements consuming resources"
* "Segment causing significant I/O"
* "Hard parsing" findings

Treat ADDM as **hypothesis generator** - validate against your application knowledge.

15. Application Developer Interpretation Patterns

Pattern A: "Database is slow" but AAS is low

* Few sessions blocked a long time (check ASH blocking chains).
* Problem may be **outside DB** (app server, network, external API).
* Look for single SQL with high **elapsed per exec** but low executions.

Pattern B: High User I/O + direct path read + parallel SQL

* Often **batch export, report, MV refresh, or CTAS**.
* Check `Module`/`Action` and job schedules (DBMS_SCHEDULER).
* Not always "bad" - may need **off-peak scheduling**.

Pattern C: enq: TX - row lock on specific table

* Map to UPDATE/DELETE/SELECT FOR UPDATE in app.
* Many users saving related entities that touch **one parent row** -> classic ORA-00060 in waits.
* Fix: shorter transactions, row-level redesign, optimistic locking, queue serialization.

Pattern D: MV refresh failing / BROKEN job

* Failed refresh can cause **repeated full refresh attempts**, heavy direct path read, high I/O.
* Stale MV data is still readable - slowness comes from **refresh competing for I/O** and related DML locks, not "stale snapshot blocking SELECT".

Pattern E: Top SQL is PL/SQL package

* SQL_ID may be wrapper; drill into **SQL ordered by CPU** and **V$SQL** for internal SQL.
* STOV example: `psto_master_bf` driving many child statements.

16. Common Wait Events Cheat Sheet

Wait Event	Wait Class	Typical root cause	Dev action
db file sequential read	User I/O	Index read, storage latency	Check plan, index, storage
db file scattered read	User I/O	Full table scan	Index, filter, partition
direct path read	User I/O	Parallel query, direct read	Schedule, limit parallelism
log file sync	Commit	Commit frequency, redo I/O	Batch commits, review txn scope
enq: TX - row lock	Concurrency	Blocking DML on same row	Txn design, lock ordering
buffer busy waits	Concurrency	Hot block	Partition, reduce contention
latch: cache buffers chains	Concurrency	Hot block in cache	Same as hot block
gc current block busy	Cluster	RAC hot row	Affinity, redesign updates
SQL*Net more data from client	Network	Client sending data slowly	JDBC batch size, network
read by other session	Concurrency	Another session changing block	Concurrent DML on same blocks
enq: TM - contention	Concurrency	DML without index on FK	Index FK columns
cursor: pin S wait on X	Concurrency	Hard parse / invalidation	Bind variables, avoid DDL peak

17. Quick Triage Checklist

Use this 10-minute pass on any AWR report:

1. **Confirm window** matches incident time and environment (DB name, RAC).
2. **Calculate AAS** = DB Time / Elapsed. Compare to CPU count.
3. **Top wait class** - note % DB Time. Pick investigation path.
4. **Top 3 wait events** - I/O vs lock vs commit vs cluster?
5. **SQL by Elapsed Time** - top 3 SQL_IDs, note Module/Action if present.
6. **SQL by Executions** - chatty patterns?
7. **Segment stats** - which tables/indexes dominate logical reads, block changes, row locks?
8. **RAC cluster waits** - if >5%, check hot objects.
9. **Load Profile** - hard parses, transactions/sec abnormal?
10. **ASH / blocking** - if lock waits, identify blocker session and SQL.

Escalate to DBA with this package

* Snapshot times, DB name, instance
* AAS, top wait class %, top 3 events
* Top 5 SQL_IDs with elapsed % and brief SQL description
* Hot segments from segment statistics
* Your hypothesis: "STOALT save hammers SITES row locks" or "MV refresh overlaps business hours"

18. Real-World Example Patterns (STOV Context)

Illustrative patterns from production triage - use as templates, not universal rules.

Example 1: MV refresh + heavy I/O

* **Symptoms:** 62% User I/O, top event `direct path read` (~59% DB time).
* **Top SQL:** Large parallel SELECT from inventory/export views.
* **Segment:** AMS antenna master MV dominates logical reads and block changes.
* **Job:** `DBMS_MVIEW.REFRESH` on schedule, status BROKEN=Y after constraint violation at source.
* **Lesson:** Broken refresh != stale reads blocked; repeated refresh attempts saturate I/O. Fix source data / rebuild MV, schedule off-peak.

Example 2: STOALT save + row locks

* **Symptoms:** `enq: TX - row lock contention` on `SITES` table, high avg wait (tens of seconds).
* **Top SQL:** `UPDATE SITES SET ... WHERE NAME = :B1` via `psto_master_bf` -> `PATO_SITES_UPD`.
* **ASH:** JDBC sessions on STOSM schema blocking each other.
* **Lesson:** Many alternatives updating same site row -> serialize or shorten transactions.

Example 3: Healthy RAC

* **Cluster waits ~0.5%** - focus on SQL and I/O, not interconnect.

19. Glossary

Term	Definition
AAS	Average Active Sessions - DB Time / Elapsed Time
ASH	Active Session History - sampled session activity
AWR	Automatic Workload Repository - snapshot repository
ADDM	Automatic Database Diagnostic Monitor - advisory from AWR
DB Time	Sum of time spent in database calls (includes waits)
Elapsed Time	Wall-clock snapshot duration
SQL_ID	Hash identifying normalized SQL text
Buffer Gets	Logical reads from buffer cache
Physical Reads	Blocks read from disk
Direct Path Read	Read bypassing buffer cache (parallel, temp, etc.)
Wait Class	Category grouping wait events
Segment	Table, index, partition, or MV
RAC	Real Application Clusters - multiple instances, shared storage
MV	Materialized View
Module / Action	End-to-end tracing tags set via DBMS_APPLICATION_INFO or JDBC

Appendix: Generating AWR Reports (DBA)

Developers typically request reports from DBA. Reference commands:

```
-- Find snapshot IDs for time range
SELECT snap_id, begin_interval_time, end_interval_time
FROM dba_hist_snapshot
WHERE begin_interval_time >= TO_DATE('2026-06-02 08:00','YYYY-MM-DD HH24:MI')
ORDER BY snap_id;

-- Generate report (run as script or via awrrpt.sql)
@?/rdbms/admin/awrrpt.sql
-- Or awrrpti.sql for specific instance in RAC
```

Document generated for internal reference. Revisit when Oracle version or workload profile changes significantly.